Enterprise NeXus — Multi-Agent AI Business Automation OS with Security Intelligence

CSE 4204 — Mobile Computing Lab | Team CSE4204-8B-T04

Week 10 Deliverable — Testing & Debugging Report

1. Introduction

This report documents the Week 10 milestone of Enterprise NeXus — Multi-Agent AI Business Automation OS with Security Intelligence: systematic testing and debugging of the full-stack, AI-integrated system. Testing covers functional behaviour, REST API contracts, the user interface, and — critically for this project — the behaviour of the AI layer, including its guarantee never to fabricate data.

2. Team Information

Name	Student ID	Role
Nazmus Sakib	11220320888	Team Leader & AI Integration Lead
Shoeb Shariar Mashuk	11220320878	Backend Developer
Most Sumiya Sanjida	11220320874	Frontend Developer — 1
Sabrina Ibrahim	11220320895	Database Manager & Frontend Developer — 2

3. Testing Strategy

Enterprise NeXus is tested with an automated Jest + Supertest suite on the backend, continuous integration on every push, and manual UI verification across devices. Tests are organised into unit tests (pure logic — auth token handling, the Gemini client, embeddings mapping) and integration tests (API endpoints exercised through the Express app with the database and AI layer mocked where appropriate).

Type	Tooling	Scope
Unit Testing	Jest	Auth tokens, AI client, RAG mapping, no-fake-data guards
API / Integration Testing	Jest + Supertest	Auth, HR, agents, executive orchestrator endpoints
UI Testing	Manual + responsive checks	Dashboard, forms, AI screens on desktop/tablet/mobile
AI Testing	Jest	Output validity, prompt effectiveness, graceful degradation
Continuous Integration	GitHub Actions	Lint + build + test on every push and PR

4. Automated Test Suites

The backend ships seven Jest suites totalling roughly 65 automated test cases. All run in CI via the "npm test" script.

Test Suite	Focus	Cases
auth.unit.test.js	Token generation, hashing, JWT edge cases	14
api.integration.test.js	Health check and HR API endpoints	14
ai.client.unit.test.js	Gemini JSON mode, retries, timeouts	9
agents.api.test.js	Agent chat endpoint behaviour	8
ai.nofakedata.unit.test.js	No fabricated data on AI outage	7
executive.orchestrator.test.js	Delegation, memory, SSE streaming	7
rag.search.test.js	match_documents mapping, RAG citations	6

5. Functional Testing

Core user journeys were verified end-to-end:

- Register → login → access protected dashboard → refresh token → logout.
- Upload a CV → AI screening report generated → candidate appears in HR list with score and recommendation.
- Create/upload an expense → anomaly severity assigned → anomaly visible on the Finance dashboard.
- Submit a support ticket → sentiment/urgency detected → grounded reply drafted → escalate/resolve.
- Ask the Executive agent a cross-domain question → delegations traced in tasks → synthesised answer returned.

6. API Testing

All REST endpoints were exercised for correct status codes, response shapes, validation, and authorisation. Protected routes reject missing or invalid tokens; role-restricted routes (for example admin-only deletes) reject unauthorised roles; and invalid payloads are rejected by Joi validators before reaching controllers. Endpoints were tested both through the automated Supertest suite and manually with Postman.

7. UI Testing

- Responsive layout verified on desktop, tablet, and mobile breakpoints.
- Loading, empty, and error states confirmed on every data-driven page.
- Form validation messages verified for all create/update forms.
- Navigation, modals, pagination, and the realtime activity feed confirmed working.

8. AI Testing

Because the AI layer is central to the product, it receives dedicated tests:

- Output validity — every agent response is validated against a Joi schema; malformed output is rejected rather than displayed.
- Prompt effectiveness — agents consistently call their data tools before answering and stay within their domain.
- Graceful degradation — when the AI client throws AiUnavailableError, controllers store a pending status instead of fabricating scores (ai.nofakedata.unit.test.js).
- Retry and timeout behaviour — transient errors (429, 5xx, network) are retried with backoff; hard timeouts are enforced.

9. Bug Report & Fixes

Representative defects found and resolved during Week 10, in the handbook's Bug / Expected / Actual format:

Bug	Expected	Actual	Fix
Login with wrong password	Rejected with 401	Returned generic 500	Standardised error middleware + 401 response
AI returns markdown-wrapped JSON	Parsed object	Parse failure	Native JSON mode with response schema
Expired refresh token reused	Rejected	New token issued	Session hash lookup + expiry check
Empty knowledge base query	Empty result handled	Unhandled error	Null-safe mapping in match_documents helper
Bulk finance upload, bad row	Row skipped/reported	Whole upload failed	Per-row validation and error reporting

10. Continuous Integration

A GitHub Actions pipeline runs on every push to main/develop and on all pull requests. It executes four jobs — backend lint, frontend lint, frontend build, and backend tests — so regressions are caught before merge. The husky pre-commit hook additionally runs linting locally.

11. Deliverables Summary

- Testing report — this document, covering functional, API, UI, and AI testing.
- Bug fix report — the defects and resolutions in Section 9.
- Automated Jest test suites (≈65 cases) and the GitHub Actions CI configuration.